Name :	Shivraj Vaibhav Jagtap
Roll No :	EN24107060
Class :	SY - AI \& DS
Subject :	Python for Visual AI

Assignment 6:

Problem Statement : You have been given a rainfall dataset of different districts in India; your task is to apply unsupervised machine learning concepts like K-means and hierarchical methods to group the different districts to apply the common policy.

K-Means Clustering

Step 1: Importing the necessary python libraries

In [ ]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

In [ ]:

```
# Load dataset
df = pd.read_csv("/content/rainfall in india 1901-2015 (1).csv")

# Display first few rows
df.head()
```

Out[ ]:

	SUBDIVISION	YEAR	JAN	FEB	MAR	APR	MAY	JUN	JUL	AUG	SEP	OCT	NOV	DEC
0	ANDAMAN & NICOBAR ISLANDS	1901	49.2	87.1	29.2	2.3	528.8	517.5	365.1	481.1	332.6	388.5	558.2	33.6
1	ANDAMAN & NICOBAR ISLANDS	1902	0.0	159.8	12.2	0.0	446.1	537.1	228.9	753.7	666.2	197.2	359.0	160.5
2	ANDAMAN & NICOBAR ISLANDS	1903	12.7	144.0	0.0	1.0	235.1	479.9	728.4	326.7	339.0	181.2	284.4	225.0

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

	SUBDIVISION	YEAR	JAN	FEB	MAR	APR	MAY	JUN	JUL	AUG	SEP	OCT	NOV	DEC
3	ANDAMAN & NICOBAR ISLANDS	1904	9.4	14.7	0.0	202.4	304.5	495.1	502.0	160.1	820.4	222.2	308.7	40.1
4	ANDAMAN & NICOBAR ISLANDS	1905	1.3	0.0	3.3	26.9	279.5	628.7	368.7	330.5	297.0	260.7	25.4	344.7

Step 2: Z-score normalization:

Normalization using Z-score: transforming data to zero mean and unit variance

$$z = \frac{x - \mu}{\sigma}$$

In [ ]:

```
# Select only rainfall columns
X = df[['JAN','FEB','MAR','APR','MAY','JUN','JUL','AUG','SEP','OCT','NOV','DEC']]

# STEP 1: Handle missing values
X = X.fillna(X.mean())

# Convert to numpy
X = X.values

#  STEP 2: Normalize safely
std = X.std(axis=0)

# Avoid division by zero
std[std == 0] = 1

X = (X - X.mean(axis=0)) / std

print("Shape:", X.shape)
print("Any NaN left:", np.isnan(X).sum())
```

```
Shape: (4116, 12)
Any NaN left: 0
```

Step 3 : Random initialization of centroids:

$$\mu_1, \mu_2, \ldots, \mu_k \in X$$

In [ ]:

```
def initialize_centroids(X, k):
    np.random.seed(42)
    indices = np.random.choice(len(X), k, replace=False)
    return X[indices]
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Step 4 : Assign each point to the nearest centroid using Euclidean distance

$$Distance = \sqrt{(X_2 - X_1)^2 + (Y_2 - Y_1)^2}$$

In [ ]:

```
def assign_clusters(X, centroids):
    clusters = []

    for point in X:
        distances = [np.linalg.norm(point - centroid) for centroid in centroids]
        cluster = np.argmin(distances)
        clusters.append(cluster)

    return np.array(clusters)
```

Step 5 : Update centroid as mean of all points in the cluster

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

In [ ]:

```
def update_centroids(X, clusters, k):
    new_centroids = []

    for i in range(k):
        points = X[clusters == i]

        # Fix empty cluster issue
        if len(points) == 0:
            new_centroids.append(X[np.random.randint(0, len(X))])
        else:
            new_centroids.append(points.mean(axis=0))

    return np.array(new_centroids)
```

Step 6 : Minimizing intra-cluster variance (objective function)

$$J = \sum_{k=1}^{K} \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

In [ ]:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
def kmeans(X, k, max_iters=100):
    centroids = initialize_centroids(X, k)

    for _ in range(max_iters):
        clusters = assign_clusters(X, centroids)
        new_centroids = update_centroids(X, clusters, k)

        # Stop if converged
        if np.allclose(centroids, new_centroids):
            break

        centroids = new_centroids

    return clusters, centroids
```

Step 7 : Iterative optimization until convergence:

$$\mu_k^{(t)} = \mu_k^{(t+1)}$$

In [ ]:

```
k = 3

clusters, centroids = kmeans(X, k)

df['KMeans_Cluster'] = clusters

print(df[['SUBDIVISION', 'YEAR', 'KMeans_Cluster']].head())
```

```
                SUBDIVISION  YEAR  KMeans_Cluster
0  ANDAMAN & NICOBAR ISLANDS  1901               1
1  ANDAMAN & NICOBAR ISLANDS  1902               1
2  ANDAMAN & NICOBAR ISLANDS  1903               1
3  ANDAMAN & NICOBAR ISLANDS  1904               1
4  ANDAMAN & NICOBAR ISLANDS  1905               1
```

In [ ]:

```
from sklearn.decomposition import PCA

# Reduce dimensions to 2D for plotting
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

# Transform centroids also
centroids_pca = pca.transform(centroids)
```

Step 8 : Finding the Distance matrix:

$$D_{ij} = \|x_i - x_j\|$$

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

In [ ]:

```
def compute_distance_matrix(X):
    n = len(X)
    dist_matrix = np.zeros((n, n))

    for i in range(n):
        for j in range(n):
            dist_matrix[i][j] = np.linalg.norm(X[i] - X[j])

    return dist_matrix
```

Hierarchical Clustering

Step 9 : Merging clusters based on minimum pairwise distance (Single Linkage)

$$d(C_i, C_j) = \min_{x \in C_i, y \in C_j} \|x - y\|$$

In [ ]:

```
def hierarchical_clustering(X, k):
    n = len(X)
    clusters = [[i] for i in range(n)]

    dist_matrix = compute_distance_matrix(X)

    while len(clusters) > k:
        min_dist = float('inf')
        merge_idx = (0, 1)

        for i in range(len(clusters)):
            for j in range(i+1, len(clusters)):
                for p in clusters[i]:
                    for q in clusters[j]:
                        if dist_matrix[p][q] < min_dist:
                            min_dist = dist_matrix[p][q]
                            merge_idx = (i, j)

        i, j = merge_idx
        clusters[i] += clusters[j]
        clusters.pop(j)

    labels = np.zeros(n)

    for idx, cluster in enumerate(clusters):
        for point in cluster:
            labels[point] = idx

    return labels
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Step 10 : Assign labels after forming k clusters

In [ ]:

```
# Take smaller sample (important)
sample_size = 150   # you can try 100–200

indices = np.random.choice(len(X), sample_size, replace=False)
X_sample = X[indices]
```

In [ ]:

```
df_sample = df.iloc[indices].copy()

h_clusters = hierarchical_clustering(X_sample, k)
df_sample['Hierarchical_Cluster'] = h_clusters

print(df_sample[['SUBDIVISION', 'YEAR', 'Hierarchical_Cluster']].head())
```

```
                          SUBDIVISION  YEAR  Hierarchical_Cluster
455   SUB HIMALAYAN WEST BENGAL & SIKKIM  1919                   0.0
1664                     HIMACHAL PRADESH  1978                   0.0
1382          HARYANA DELHI & CHANDIGARH  1926                   0.0
1022                   EAST UTTAR PRADESH  1911                   0.0
2922                              VIDARBHA  1971                   0.0
```

Step 11 : Visual representation of clustered data in feature space

In [ ]:

```
plt.figure(figsize=(8,5))

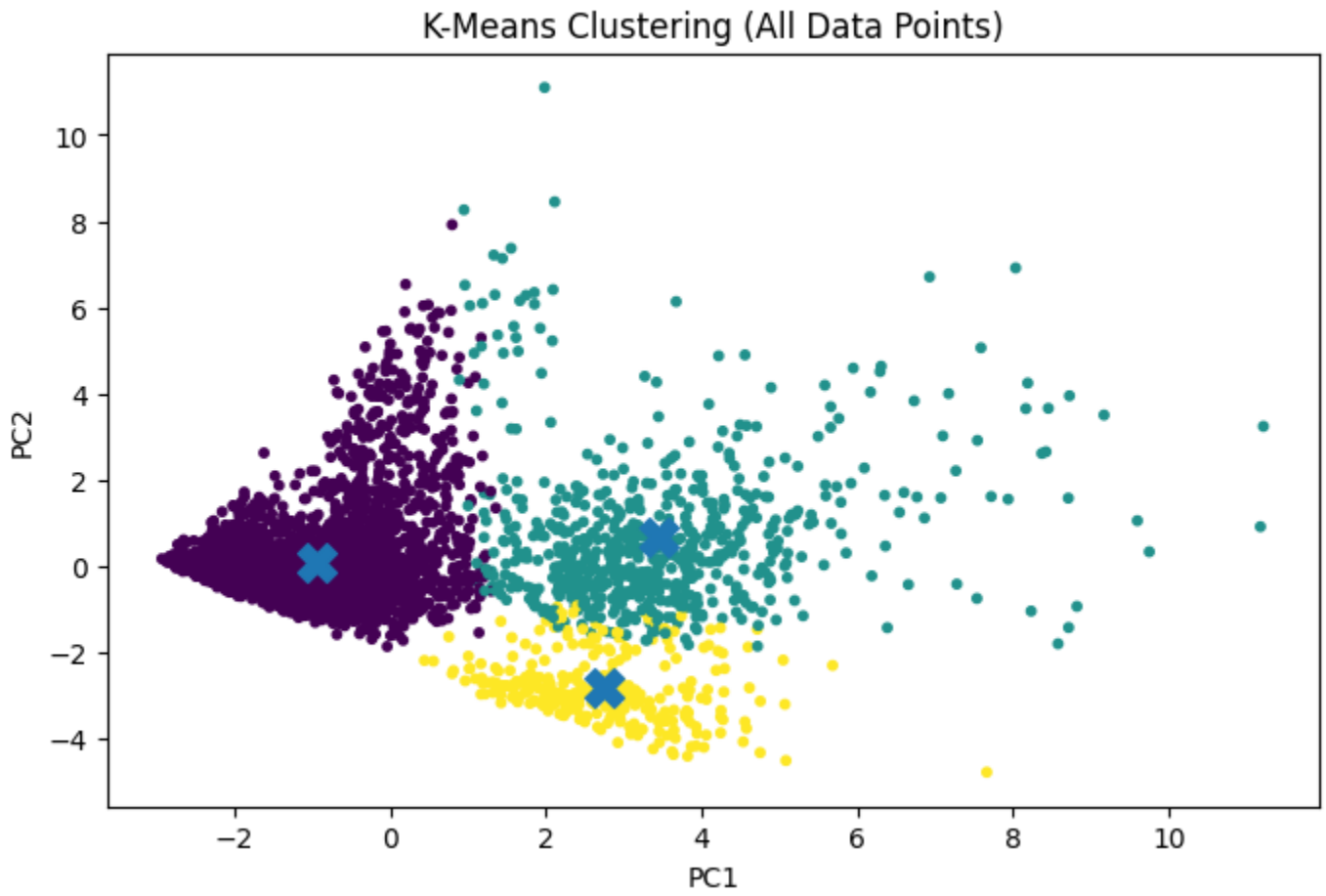
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, s=10)

plt.scatter(centroids_pca[:, 0], centroids_pca[:, 1], marker='X', s=200)

plt.title("K-Means Clustering (All Data Points)")
plt.xlabel("PC1")
plt.ylabel("PC2")

plt.show()
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF



Step 12 : Final clustering result: mapping subdivisions to clusters

In [ ]:

```
plt.figure(figsize=(14,6))

# ------------------ K-MEANS ------------------
plt.subplot(1, 2, 1)

plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, s=10)

# Centroids
plt.scatter(centroids_pca[:, 0], centroids_pca[:, 1], marker='X', s=200)

plt.title("K-Means Clustering (Full Data)")
plt.xlabel("PC1")
plt.ylabel("PC2")


# ------------------ HIERARCHICAL ------------------

# Apply PCA on sample data
X_sample_pca = pca.transform(X_sample)

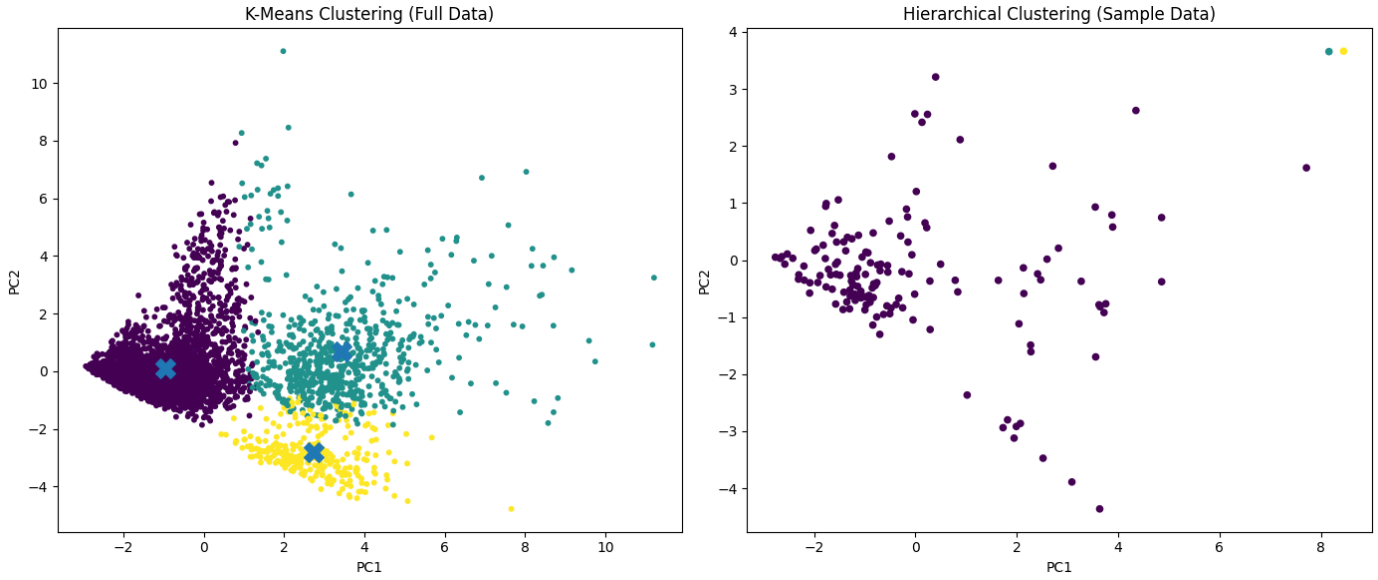
plt.subplot(1, 2, 2)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
plt.scatter(X_sample_pca[:, 0], X_sample_pca[:, 1], c=h_clusters, s=20)

plt.title("Hierarchical Clustering (Sample Data)")
plt.xlabel("PC1")
plt.ylabel("PC2")


plt.tight_layout()
plt.show()
```



Conclusion

This study applied unsupervised machine learning techniques to analyze rainfall patterns across regions of India using K-Means and Hierarchical Clustering. K-Means was used on the full dataset for its efficiency, while Hierarchical Clustering was applied to a sample due to its higher computational cost. The results show that regions with similar rainfall patterns can be effectively grouped, aiding in policy-making, agriculture, and water management. The study also emphasizes choosing algorithms based on dataset size and computational limits.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF